Design 1: InfractionTrigger Interface and EnforcerBehaviour Abstract Class

This penalty system implements two distinct abstract concepts: the `InfractionTrigger` interface, used to detect infractions, and the `EnforcerBehaviour` abstract class, used for the `EnforcerBot`. Each abstract concept is implemented by three concrete classes.

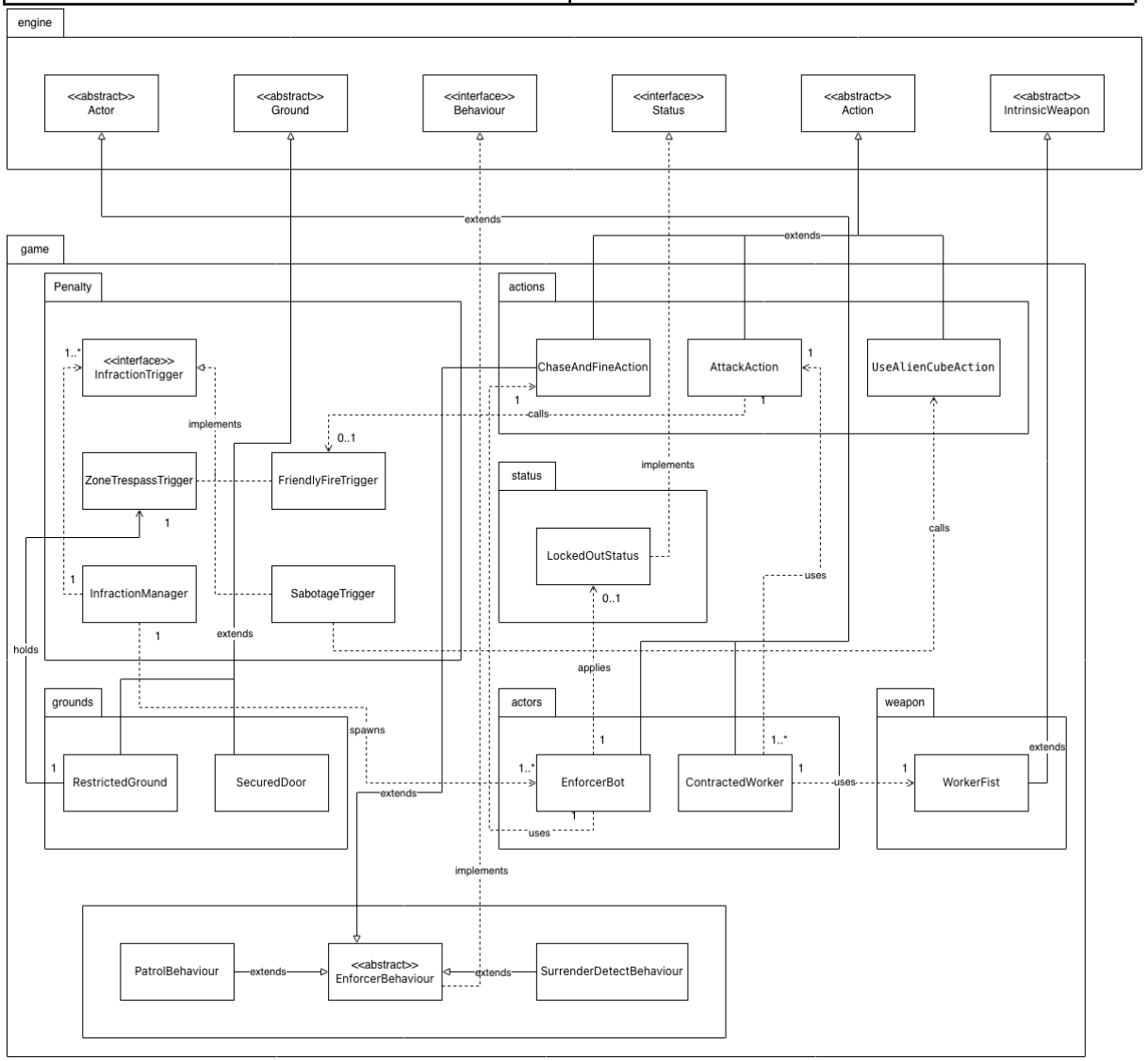
pros	cons
In accordance with the DRY principle, the shared target fields and the `operate()` contract have been centralized in `EnforcerBehaviour` and are not repeated across the three behavior subclasses.	This requires two separate abstractions, which increases the number of classes and adds some structural complexity.
Following the OCP principle, we can add new violation types by implementing `InfractionTrigger` without modifying existing triggers or managers.	The relationship between `EnforcerBehaviour` and `ChaseAndFineAction` (located in another package) requires cross-package imports.
In following the SRP principle, each trigger class detects only one type of violation, and each behavior class defines only one bot action.	
Following the DIP principle, the `InfractionManager` depends on the `InfractionTrigger` interface rather than a specific trigger class, while the `EnforcerBot` depends on `EnforcerBehaviour` rather than a specific behavior class.	

Design 2: Single Abstract Class with Embedded Trigger Logic

An abstract class called `InfractionHandler` combines both violation detection and bot behavior functions. Each child class (such as `TrespassHandler` and `FriendlyFireHandler`) directly handles its own trigger condition checks and generates its own `EnforcerBot`.

pros	cons
A simpler class structure, requiring only one abstract layer.	This violates the SRP principle,each class is responsible for both detecting violations and controlling the robot's behavior, even though these are two completely distinct goals.
Each handler is self-contained and easy to understand when read on its own.	This violates the DRY principle,the robot's generation and path planning logic will be repeated in every process child class.
	This violates the OCP principle,adding new

	robot behaviors (such as patrolling) requires modifying each existing implementation rather than creating a new child class.
	This violates the DIP principle,the `InfractionManager` will depend on a specific handler class rather than an abstract class.



I chose Design 1. Although it introduces two abstract classes instead of one, it separates infraction detection (InfractionTrigger) from bot behavior (EnforcerBehaviour), significantly improving maintainability and extensibility. This design follows the DRY, OCP, SRP, and DIP principles. If new infraction types need to be added in the future, simply create a new class that implements the `InfractionTrigger` interface; there is no need to modify existing triggers or managers. Similarly, new robot behaviors can be added by extending `EnforcerBehaviour` without changing existing behavior classes.